

Amar Telidji University Laghouat

Department of Computer Science

Level: 4th Year Engineering

Course: Advanced Network

Lab 04: Program Security Report

Supervised by:

Mr. Mohamed Lahcen BENSaad

Prepared by:

Smail MERSAD

Academic Year: 2025 / 2026

Introduction:

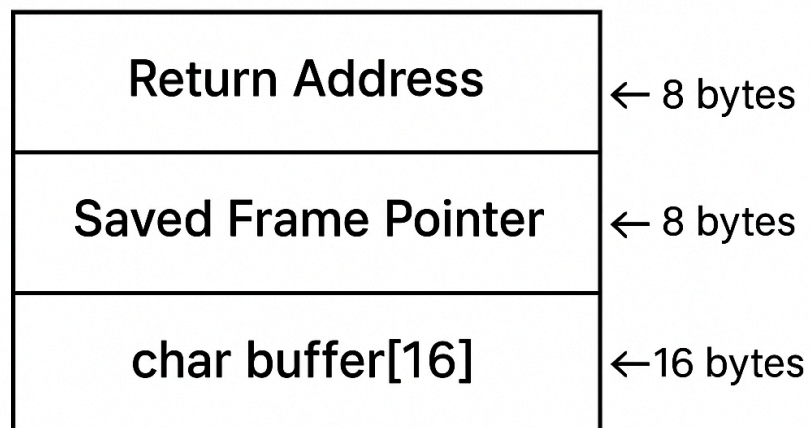
This lab explores common program security vulnerabilities including buffer overflows, SQL injection, unsafe memory usage, and common web attacks such as Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF).

The objective is to understand how these vulnerabilities occur, how attackers exploit them, and how secure coding practices can prevent them.

Part 1: Buffer Overflow Investigation

1. Stack Frame Analysis

Higher Memory Addresses



Lower Memory Addresses

The stack frame of `vulnerable_function` contains a fixed-size buffer of 16 bytes, followed by the saved frame pointer (SFP) and the return address. Since `strcpy` does not check input length, excess data can overwrite adjacent stack memory.

2. Minimum Characters to Overwrite Return Address

- Buffer size = **16 bytes**
- Saved Frame Pointer = **8 bytes**
- Total before return address = **24 bytes**

Minimum characters required to start overwriting return address:

16 + 8 = 24 bytes

The 25th byte begins overwriting the return address.

3. Program Crash Explanation

When the input exceeds the buffer size, `strcpy` continues copying data past the allocated memory. This overwrites the saved frame pointer and return address.

Once the function returns, the corrupted return address causes the program to jump to an invalid memory location, leading to a segmentation fault.

Part 2: Secure Coding & Input Validation

1. SQL Injection Explanation

Attack example:

```
' OR '1'='1'
```

Resulting query:

```
SELECT * FROM users WHERE username = '' OR '1'='1'
```

An attacker can manipulate the **username** input to alter the SQL query logic. This allows bypassing authentication or retrieving all user records because user input is directly concatenated into the SQL command.

2. Secure Version Using Prepared Statements

```
#include <stdio.h>

#include <ctype.h>

#include <sqlite3.h>

int is_valid_username(const char *username) {

    for (int i = 0; username[i]; i++) {

        if (!isalnum(username[i])) {

            return 0;

        }

    }

}
```

```
}

return 1;

}

void get_user_info(sqlite3 *db, char *username) {

    if (!is_valid_username(username)) {

        printf("Invalid username\n");

        return;

    }

    sqlite3_stmt *stmt;

    const char *sql = "SELECT * FROM users WHERE username = ?";

    if (sqlite3_prepare_v2(db, sql, -1, &stmt, NULL) != SQLITE_OK) {

        return;

    }

    sqlite3_bind_text(stmt, 1, username, -1, SQLITE_STATIC);

    sqlite3_step(stmt);

    sqlite3_finalize(stmt);

}
```

3. Why This Prevents SQL Injection

Prepared statements separate SQL code from user data. The database engine treats user input strictly as data, even if it contains SQL keywords.

Input validation further restricts usernames to alphanumeric characters, reducing attack surface.

Part 3: Static Analysis & Code Review

Vulnerability 1: Integer Overflow

```
int size = count * 20;
```

Why it's dangerous:

If **count** is very large, integer overflow can occur, resulting in a smaller-than-expected allocation. This can lead to buffer overflow when data is written.

Fix:

```
if (count <= 0 || count > INT_MAX / 20) {  
  
    return;  
  
}
```

Vulnerability 2: Format String Vulnerability

```
printf(buffer);
```

Why it's dangerous:

If **buffer** contains format specifiers (e.g., **%x**), attackers can read memory or crash the program.

Fix:

```
printf("%s", buffer);
```

Part 4: Web Security (XSS & CSRF)

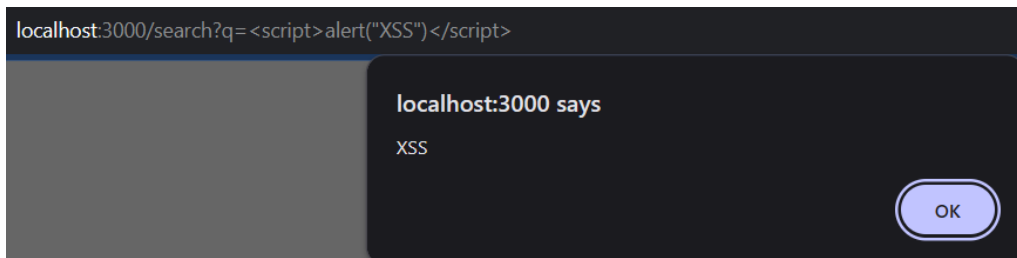
Task 1: Reflected XSS

XSS Payload

```
<script>alert("XSS")</script>
```

Result:

When this payload is passed as the **q** parameter, the browser interprets it as JavaScript and executes it, displaying an alert box.



Secure Code Fix

```
app.get('/search', (req, res) => {  
  
    let query = req.query.q;  
  
    if (query) {  
  
        query = query  
  
            .replace(/&/g, "&amp;")  
  
            .replace(/</g, "&lt;")  
  
            .replace(/>/g, "&gt;")  
  
            .replace(/"/g, "&quot;")  
  
            .replace(/'/g, "&#x27;");  
  
    }  
  
    res.send('You searched for: ' + query);  
});
```

Explanation:

The vulnerability occurs because user input is directly embedded into the HTML response without validation or sanitization. This allows attackers to inject executable JavaScript code.

The issue is fixed by manually escaping special HTML characters such as `<`, `>`, `"`, `'`, and `&`. These characters are replaced with their corresponding HTML entities before being sent to the browser.

As a result, the browser renders the input as plain text instead of executing it as code, effectively preventing reflected XSS attacks.

Task 2: CSRF

CSRF Attack HTML

```
<form action="https://bank.com/transfer_funds" method="POST">

  <input type="hidden" name="to_account" value="ATTACKER123">

  <input type="hidden" name="amount" value="1000">

</form>

<script>

  document.forms[0].submit();

</script>
```

CSRF Defense Mechanism

Defense Name: **CSRF Token**

Explanation:

A CSRF token is a unique, unpredictable value generated by the server and embedded in each form. The server verifies this token upon form submission. Since an attacker cannot guess or access this token, forged requests from malicious websites are rejected.